Project 2 Architecture & Specification Blueprint (FairShare)

Zichen Tian, Zheqi Lv
https://github.com/Lildonk/FairShare

1. Executive Summary & Problem Formulation

1.1 Project Overview

FairShare is a full-stack, multi-party expense sharing and automated debt simplification web application engineered for roommates, travel groups, and event organizers. Moving beyond basic pairwise calculators, FairShare resolves collective balance tracking through an algorithmic graph-based debt minimization engine, multi-modal split mechanisms (equal, exact, percentage, itemized with tax/tip pro-rating), and role-based access control (RBAC). The platform is powered by Next.js 14 App Router, PostgreSQL with Prisma ORM, JWT-based user isolation, comprehensive Vitest/Playwright test suites, and public OpenAPI/Swagger endpoints.

1.2 Grounded User Pain Points (Mom-Test Validated)

- **N-Way Settlement Overhead**: In typical groups of 4–8 individuals, members pay for overlapping group costs (accommodations, rental cars, shared groceries, group meals). Without coordination, this creates up to $\frac{N(N-1)}{2}$ chaotic bilateral debts, resulting in confusion and payment drop-off.
- **Transaction Fee & Friction Waste**: Direct pairwise settlement requires dozens of manual peer-to-peer transfers (e.g., 15 transactions for 6 people). Users require algorithmic debt simplification to compress transaction graphs down to $N-1$ net transactions (e.g., 3–4 transfers) without penny-rounding discrepancy.
- **Proportional Tax & Tip Distortion**: Shared dining or supply bills are rarely split equally. Itemized splits that ignore proportional distribution of local taxes, service tips, and shared appetizers lead to calculation errors and social friction.
- **Zero-Friction Access & Audit Transparency**: Group members resist installing intrusive third-party mobile apps or linking bank credentials via Plaid. They demand a transparent, web-accessible system with exportable audit trails and dispute resolution flags.

---

2. Technical Stack & System Architecture

Architecture Layer	Technology Selection	Architectural Scope & Rationale
Frontend Framework	**Next.js 14+ (App Router)**	Server Components (RSC) for fast initial group hydration; Client Components for reactive split interfaces.
Styling & UI Components	**TailwindCSS & Lucide React**	Mobile-first responsive layouts with clean visual transaction graphs and split breakdown chips.
Backend & API Layer	**Next.js Route Handlers**	RESTful modular endpoints for group administration, expense ingestion, and settlement generation.
Database & ORM	**PostgreSQL + Prisma ORM**	Strict relational consistency hosted on Neon/Supabase with declarative schema migrations.
Authentication & RBAC	**JWT + Bcrypt.js**	Stateless Bearer token authentication with user-isolated queries and role-guarded routes.
State Management	**Zustand + React Hooks**	Real-time split preview calculations, dynamic participant toggles, and optimistic balance updates.
Unit & TDD Testing	**Vitest + RTL**	Sub-second test execution verifying debt graph algorithms and cents-precision math with Red-to-Green Git commits.
End-to-End Testing	**Playwright**	Headless browser testing of the critical user journey: Register -> Create Group ->

Architecture Layer	Technology Selection	Architectural Scope & Rationale
		Split Expense -> Simplify Debts.
API Documentation	**Swagger UI / OpenAPI 3.0**	Interactive endpoint documentation and schema explorer hosted at `/api/docs`.
CI/CD Pipeline	**GitHub Actions**	Automated workflow executing TypeScript type checking, ESLint, and Vitest test suites on every PR.

---

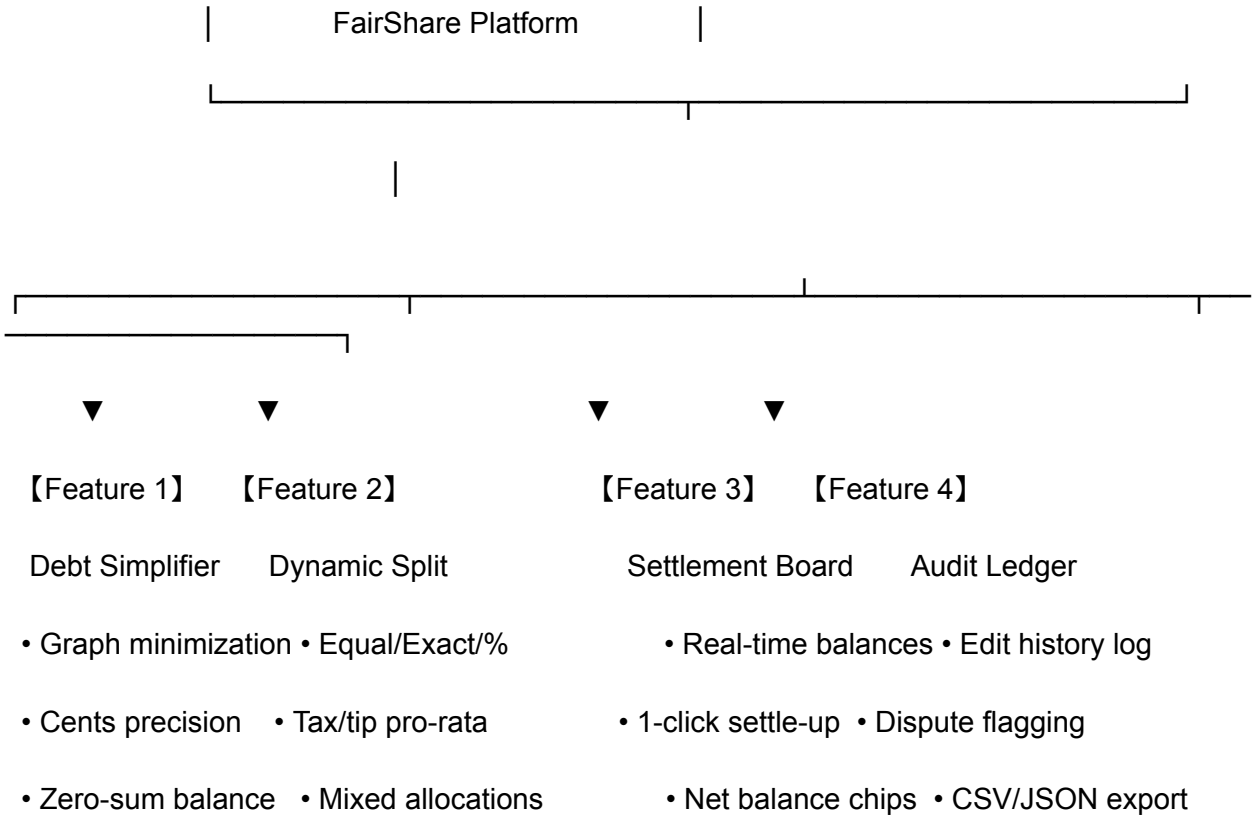
3. User Roles & Permission Model (RBAC)

FairShare defines three explicit access tiers fulfilling the `3+ distinct features/roles` specification:

1. **Group Member (Authenticated Regular User - `MEMBER`)**:
- Create groups or join existing groups via secure invite tokens.
- Add, view, and edit shared expenses across multiple split configurations.
- Mark individual debts as settled and view personal net balances.
2. **Group Admin / Auditor (Role: `ADMIN`)**:
- Protected access to `/admin` routes via JWT role middleware.
- Lock group settlement cycles, archive completed trip ledgers, and manage group member rosters.
- Manage global currency templates and resolve disputed/flagged transactions.
3. **Guest / Public API Consumer (Unauthenticated)**:
- Public access to interactive OpenAPI/Swagger documentation (`/api/docs`).
- Unauthenticated consumption of utility endpoints: invoke the standalone debt simplification algorithm (`POST /api/v1/simplify`) or test split calculations (`POST /api/v1/split/calculate`).

---

4. Core Feature Breakdown & Acceptance Criteria



Feature 1: Debt Graph Simplification Engine (TDD Target / HW3 Comparison)

- **Scope**: Algorithmic engine computing net financial positions across all members and executing a greedy two-pointer graph minimization algorithm to minimize total settlement transactions.
- **Acceptance Criteria**:
- Net Balance Computation: For each participant i, computes $\text{Net}_i = \sum \text{Paid}_i - \sum \text{Owed}_i$.
- Cents Precision Standard: All internal arithmetic uses integer cents to prevent IEEE-754 floating-point truncation ($0.01 bug).
- Graph Minimization: Matches the largest creditor with the largest debtor iteratively, reducing $O(N^2)$ cross-debts to at most $N-1$ optimal transfers.
- Zero-Sum Verification: Validates that $\sum \text{Net}_i = 0$ prior to solving; rejects unbalanced input matrices.
- Edge Cases: Gracefully handles zero-balance participants, identical mutual debts, single-debtor setups, and empty transaction lists.
- 100% Vitest coverage with Git commits demonstrating failing tests before passing implementations.

Feature 2: Multi-Modal Dynamic Expense Split Engine

- **Scope**: Flexible expense intake module supporting multiple bill distribution paradigms.
- **Acceptance Criteria**:
- **Equal Split**: Even division with remainder cents assigned deterministically to the payer.
- **Exact Amount Split**: Validates that the sum of individual custom allocations equals the total receipt amount.
- **Percentage Split**: Verifies allocations sum to exactly 100.0% within a 0.01% tolerance.
- **Itemized Split with Tax & Tip**: Pro-rates shared tax, service charges, and tips across itemized sub-totals.

Feature 3: Interactive Group Settlement & Balance Board

- **Scope**: Real-time group dashboard visualizing collective financial health and pending transfer requests.
- **Acceptance Criteria**:
- Displays high-contrast balance chips for each member: green for net creditors (gets back $X), red for net debtors (owes$Y).
- Lists generated optimal transfer instructions (e.g., "Alice pays Charlie $42.50").
- 1-click "Record Payment" button updating settlement status with optimistic UI updates.

Feature 4: Audit Trail, Receipt Ledger & Dispute Flagging

- **Scope**: Complete chronological activity ledger supporting accountability and dispute resolution.
- **Acceptance Criteria**:
- Maintains immutable record of expense additions, updates, and deletions with timestamps.
- Provides a "Dispute / Review Needed" toggle for contested transactions.
- 1-click export of group settlement data to JSON and CSV formats.

---

5. Relational Database Schema Design (Prisma)

```
datasource db {

  provider = "postgresql"

  url      = env("DATABASE_URL")

}

generator client {
```

```
  provider = "prisma-client-js"
}

enum Role {
  MEMBER
  ADMIN
}

enum SplitType {
  EQUAL
  EXACT
  PERCENTAGE
  ITEMIZED
}

enum SettlementStatus {
  PENDING
  CONFIRMED
  DISPUTED
}

model User {
  id           String        @id @default(uuid())
  email        String        @unique
  passwordHash String
  name         String
```

```
  role          Role          @default(MEMBER)
  memberships   GroupMember[]
  paidExpenses  Expense[]     @relation("ExpensePayer")
  splits        ExpenseSplit[]
  createdAt     DateTime      @default(now())
  updatedAt     DateTime      @updatedAt
}

model Group {
  id          String         @id @default(uuid())
  name        String
  description String?
  currency    String         @default("USD")
  inviteCode  String         @unique @default(uuid())
  isArchived  Boolean        @default(false)
  members     GroupMember[]
  expenses    Expense[]
  settlements Settlement[]
  createdAt   DateTime       @default(now())
  updatedAt   DateTime       @updatedAt
}

model GroupMember {
  id        String   @id @default(uuid())
```

```
  userId    String
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  groupId   String
  group     Group    @relation(fields: [groupId], references: [id], onDelete: Cascade)
  joinedAt  DateTime @default(now())
  @@unique([userId, groupId])
}
model Expense {
  id          String        @id @default(uuid())
  groupId     String
  group       Group         @relation(fields: [groupId], references: [id], onDelete: Cascade)
  payerId     String
  payer       User          @relation("ExpensePayer", fields: [payerId], references: [id], onDelete:
Cascade)
  title       String
  amountCents Int
  splitType   SplitType     @default(EQUAL)
  category    String        @default("General")
  isDisputed  Boolean       @default(false)
  receiptUrl  String?
  splits      ExpenseSplit[]
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @updatedAt
```

```
}

model ExpenseSplit {
  id          String   @id @default(uuid())
  expenseId   String
  expense     Expense  @relation(fields: [expenseId], references: [id], onDelete: Cascade)
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  owedCents   Int
  allocatedPct Float?
}

model Settlement {
  id          String           @id @default(uuid())
  groupId     String
  group       Group            @relation(fields: [groupId], references: [id], onDelete: Cascade)
  fromUserId  String
  toUserId    String
  amountCents Int
  status      SettlementStatus @default(PENDING)
  notes       String?
  settledAt   DateTime?
  createdAt   DateTime         @default(now())
}
```

---

6. Public REST API & Swagger Specification

Base URL: `/api/v1`

Interactive Documentation: `/api/docs`

HTTP Method	Route	Access Level	Description
`POST`	`/api/v1/auth/register`	Public	Register new user account with hashed password.
`POST`	`/api/v1/auth/login`	Public	Authenticate user credentials and return signed JWT token.
`POST`	`/api/v1/simplify`	**Public**	Pure algorithmic endpoint: accepts raw debts, returns simplified transfer graph.
`POST`	`/api/v1/split/calculate`	**Public**	Pure calculation utility: computes itemized split distributions with tax/tip.
`GET`	`/api/v1/groups`	Authenticated	Retrieve all groups the current user belongs to.
`POST`	`/api/v1/groups`	Authenticated	Create a new expense sharing group.
`POST`	`/api/v1/groups/:id/expenses`	Authenticated	Create an expense with specified split type and allocations.

HTTP Method	Route	Access Level	Description
`GET`	`/api/v1/groups/:id/settlements`	Authenticated	Calculate and fetch real-time simplified settlement instructions.
`PATCH`	`/api/v1/settlements/:id`	Authenticated	Update settlement status (`PENDING` -> `CONFIRMED` or `DISPUTED`).
`GET`	`/api/v1/admin/groups`	**Admin Only**	Global platform overview of active groups and dispute logs.
`PATCH`	`/api/v1/admin/groups/:id/archive`	**Admin Only**	Archive group and freeze ledger against further modifications.

---

7. Agile Development & Engineering Standards

7.1 Git Branching & Commit Conventions

- **Branch Format**: `feature/<issue-number>-<short-description>` (e.g., `feature/4-debt-simplifier-engine`).
- **Commit Standards**: Conventional Commits linking issues:
- Red Phase: `test(engine): add failing unit tests for cyclic debt resolution (#4)`
- Green Phase: `feat(engine): implement greedy two-pointer debt simplification (#4)`
- Refactor: `refactor(engine): optimize balance normalization into integer cents (#4)`

7.2 Pull Request & Code Review Governance

- Direct pushes to `main` are strictly blocked.
- Every PR must link to its corresponding GitHub Issue using `Resolves #<issue-number>`.

- **Mandatory Review Rule**: A PR can only be merged after receiving a formal approval and at least one substantive code comment from the partner (ensuring each member reaches 5+ verified reviews).

7.3 Test-Driven Development (TDD) Protocol

- Tests are committed in a failing state (Red) prior to writing implementation code (Green).
- Core calculation engines (graph debt simplification, tax/tip pro-rating) require 100% test coverage in Vitest.

---

8. Sprints & Milestones Delivery Roadmap

Week 6 ──► Sprint 0 / HW3 (Oct 22 - Oct 28)

├── Scaffolding, GitHub Board, Rules Engine (.antigravityrules)

└── A/B Prompt Testing & First TDD PR (Issue #4)

Week 7 ──► Sprint 1 Core (Oct 28 - Nov 4)

├── PostgreSQL Prisma Schema, Seed Data, JWT Auth

└── Expense Creation UI & Dynamic Split Engine

Week 8 ──► Sprint 2 Full-Stack (Nov 4 - Nov 11)

├── Interactive Settlement Dashboard & Visual Balance Graph

└── Public REST API, Swagger Docs & Admin Portal

Week 9 ──► Final Delivery (Nov 11 - Nov 12)

├── Playwright E2E Test Suite & Vercel Deployment

└── 3-5 Min Demo Video, Contribution Log & Final Submission

8.1 Sprint 1 Deliverables (Milestone: Nov 4, 2026)

- **Issue #1 (`Zheqi Lv`)**: Next.js 14, TailwindCSS, Vitest, and GitHub Actions CI workflow.
- **Issue #2 (`Zheqi Lv`)**: PostgreSQL schema, Prisma migrations, and sample seed data.

- **Issue #3 (`Zheqi Lv`)**: JWT authentication, password hashing, and route protection middleware.
- **Issue #4 (`Zichen Tian`)**: Debt simplification engine, cents conversion, and TDD Vitest test suite.
- **Issue #5 (`Zichen Tian`)**: Expense creation modal supporting Equal, Exact, and Percentage splits.
- **Issue #6 (`Zichen Tian`)**: Pro-rated itemized bill split calculation engine with tax/tip logic.

8.2 Sprint 2 Deliverables (Milestone: Nov 11, 2026)

- **Issue #7 (`Zichen Tian`)**: Interactive group settlement dashboard and member balance chips.
- **Issue #8 (`Zichen Tian`)**: Visual transaction graph visualizer and audit dispute ledger.
- **Issue #9 (`Zheqi Lv`)**: Public REST API endpoints with interactive Swagger/OpenAPI documentation.
- **Issue #10 (`Zheqi Lv`)**: Playwright E2E test suite covering full user journey.
- **Issue #11 (`Zheqi Lv`)**: Role-protected `/admin` portal for group archiving and dispute management.